OTA Ingestion Framework

Author: Daniel Wondyfiraw · Updated: 30 October 2026

Prepare URLs → test → collect → export CSVs → load BigQuery.

User guide: installation, collection and BigQuery loading.

1. Clone the repository and install

Use Python 3.12 on Linux. Paths and environment activation can differ between machines.

If you get blocked, contact Daniel in the team WhatsApp group with the command and error message. Keep API keys and credential-file contents private.

Open a terminal:

```
git clone https://github.com/open-trace/data-team.git
cd data-team/data-eng/ota_ingestion_framework
python3.12 -m venv ota_venv
source ota_venv/bin/activate
python -m pip install -e .
ota-batch --help
```

Already cloned the repository? Open its data-eng/ota_ingestion_framework folder instead. Activate ota_venv whenever you open a new terminal.

For PDF text extraction, install Poppler:

```
sudo apt-get install poppler-utils
```

2. Choose your input

Choose A, B or C. CSV and Excel references contain URLs; they are not price spreadsheets imported directly. Run these commands from the framework folder.

A. Batch from CSV, Excel or text references

Place reference files in ingestion_points/raw/, then run:

```
ota-batch --workers 4 --max-seconds 300 --timeout 15
```

This prepares, tests, collects and exports. Continue at Step 3.

B. One URL, entered directly

Set a separate output folder, supply the URL and list its generated source ID:

```
export OTA_STORAGE_DIR="$PWD/data/single-source"
ota-prepare --url "https://durbanmarkets.durban.gov.za/"
ota-test --list
```

You do not need to invent an ID. Enter the ID displayed by the list:

```
read -r -p "Source ID from the list: " SOURCE_ID
ota-test --source-id "$SOURCE_ID" --max-seconds 60
ota-collect --source-id "$SOURCE_ID" --max-seconds 300
ota-export
```

C. One selected CSV or Excel reference file

A CSV can contain:

```
source_id,display_name,source_url
za_durban_market_fresh,Durban market,https://durbanmarkets.durban.gov.za/
```

Save it as ingestion_points/raw/my_sources.csv, then run:

```
export OTA_STORAGE_DIR="$PWD/data/selected-file"
ota-prepare --file ingestion_points/raw/my_sources.csv
ota-test --list
read -r -p "Source ID from the list: " SOURCE_ID
ota-test --source-id "$SOURCE_ID" --max-seconds 60
ota-collect --source-id "$SOURCE_ID" --max-seconds 300
ota-export
```

For Excel, use ota-prepare --file path/to/sources.xlsx instead. A selected file can contain one or many URLs. Direct URL preparation automatically names the source; existing source-specific rules can be selected with an explicit ID:

```
ota-prepare --url "https://durbanmarkets.durban.gov.za/" \
  --source-id za_durban_market_fresh
```

Use the displayed ID for the remaining commands. New websites can require an extraction adapter. Do not run ota-batch after direct preparation expecting a one-source run: it scans the full reference folder again.

3. Review the export

Open data/exports/latest/, or $OTA_STORAGE_DIR/data/exports/latest/ when using custom storage. Inspect the CSVs, manifest and source reports before loading.

--workers: concurrent hosts; each host's sources are processed sequentially.

--max-seconds: time budget per source operation, not total batch duration.

--timeout: request timeout during assessment.

--recheck: refresh cached assessments in a batch.

A connection pass confirms access. PARTIAL means checks or collection remain incomplete. Failures are recorded while other batch sources continue. API paging follows the source's configured contract. Exports can reuse an earlier useful collection after a newer failed attempt; the manifest identifies this.

4. Prepare the BigQuery package

OTA uses project opentrace-prod-5ga4, dataset landing and location europe-west3. Use an account authorized for this destination:

```
export BQ_PROJECT="opentrace-prod-5ga4"
export BQ_DATASET_LANDING="landing"
export DBT_BIGQUERY_LOCATION="europe-west3"
ota-prepare-load --project "$BQ_PROJECT" \
  --dataset "$BQ_DATASET_LANDING" \
  --location "$DBT_BIGQUERY_LOCATION"
ota-load
```

Review table_plan.csv, schemas and 00_landing_tables.sql under data/bigquery/latest/ in the active storage folder. These commands create and validate local files; they do not load cloud tables.

5. Check and load BigQuery

Supply your own authorized Google credentials. Enter the absolute path to your credential JSON file when prompted:

```
read -r -p "Google credential JSON path: " GOOGLE_APPLICATION_CREDENTIALS
export GOOGLE_APPLICATION_CREDENTIALS
ota-load --check
```

Resolve reported errors, then execute the reviewed first load:

```
ota-load --execute
```

```
```

Check data/bigquery/load_report.json in the active storage folder. Confirm each table's result and reconciled count. To retry, preserve the package and state and rerun ota-load --execute. Loading a new daily snapshot into populated tables requires an incremental loading policy. Existing OTA tables are not overwritten by this loader. Business transformations and BI follow landing.

API credentials

For sources requiring a key, create your local file:

```
cp config/tls/api_key.example.json config/tls/api_key.json
```

Edit the ignored file with your own credentials:

```
{
  "API_NINJAS": {"api_key": ""},
  "API_OTHER": {"api_key": ""}
}
```

Enter your key in the relevant api_key field. API_OTHER is an example name. Each API needs a connector defining its host and authentication method. The API_NINJAS_KEY environment variable takes precedence over the file. Keep credentials out of Git. Public sources may need no key.

Folder structure

```
ota_ingestion_framework/
├── README.md                    # Setup and commands
├── pyproject.toml               # Package and ota-* commands
├── requirements.txt             # Python dependencies
├── Dockerfile                   # Collection container
├── railway.json                 # Collection schedule
├── config/tls/                  # Public certificates and local API keys
├── ingestion_points/
│   ├── raw/                     # Input references containing source URLs
│   ├── testing_rules.json       # Source request and extraction settings
│   └── generated/               # Generated URLs, assessments and profiles
├── src/                         # Commands and common helpers
├── tests/                       # Offline tests
├── docs/
│   └── OTA_Ingestion_Framework_User_Guide.pdf  # Published walkthrough
├── data/                        # Generated locally
│   ├── raw/                     # Responses, reports and checkpoints
│   ├── collected/               # Extracted records
│   ├── exports/latest/          # CSV handover and metadata
│   └── bigquery/                # Schemas, DDL, records and loading state
├── archive/                     # Local retired files, outside the pipeline
└── ota_venv/                     # Local Python environment
```

You provide reference inputs and credentials. Commands generate inventories, profiles, records, exports and loading packages. Generated outputs, credentials, archives and environments are ignored by Git. docs/ contains the published PDF walkthrough. Archive contents are outside the collection workflow and are not scanned for source URLs by default.

OTA_REFERENCE_DIR optionally selects another reference folder. OTA_STORAGE_DIR puts generated outputs under its data/ and state/ folders. To return to the default folders:

```
unset OTA_REFERENCE_DIR OTA_STORAGE_DIR
```

How it connects

```
URL or file → prepare → test → collect → CSV export
                                            ↓
BigQuery landing ← load/reconcile ← check ← schemas and load package
```

Daniel Wondyfiraw · OpenTrace Africa

ota-batch coordinates preparation through export. BigQuery loading runs separately after review. Original responses and metadata preserve source lineage.

Run tests

All automated tests:

```
python -m unittest discover -s tests -q
```

Only the four end-to-end workflow checks:

```
python -m unittest discover -s tests -p test_pipeline_workflows.py -v
```

Workflow checks use temporary synthetic data and simulated HTTP/BigQuery services. They cover URL and file inputs, batch formats, API paging, access failures, exports and load recovery. Live connectivity and cloud permissions are checked separately by ota-test and ota-load --check.

Next steps

Railway: validate the collection container and persistent storage, then enable the scheduled job. Use service root data-team/data-eng/ota_ingestion_framework and a volume at /storage. Supply API keys as service variables. The configured collection/export schedule is 03:00 UTC; BigQuery loading remains separate.

v2.0: release the transformation workflow, including dbt models and data checks, after mart and BI testing. Reporting tables and BI examples will follow the agreed business rules.

v3.0 (proposed): evaluate Airflow for orchestration, Airbyte for supported connectors, and cloud infrastructure for deployment, scheduling, monitoring and credential management. Final tools and scope will follow operational needs.